In [1]:
```
"""
Continuous Bag of Words (CBOW) model from scratch with NumPy.

Stages:
  a. Data preparation
  b. Generate training data
  c. Train model
  d. Output
"""
import re
import numpy as np

np.random.seed(42)
```

In [2]:
```
# ---------------------------------------------------------------
# a. DATA PREPARATION
# ---------------------------------------------------------------
corpus = """
The quick brown fox jumps over the lazy dog.
The dog barks at the fox and the fox runs away.
A quick brown dog jumps over the lazy fox.
Natural language processing helps computers understand human language.
Word embeddings capture the meaning of words in vectors.
The model learns word vectors by predicting a word from its context.
Computers learn language from large amounts of text.
The cat and the dog are friends and the cat likes the dog.
"""


def preprocess(text):
    text = text.lower()
    text = re.sub(r"[^a-z\s]", " ", text)      # drop punctuation
    return text.split()                         # tokenise


tokens = preprocess(corpus)
vocab = sorted(set(tokens))
V = len(vocab)
word2idx = {w: i for i, w in enumerate(vocab)}
idx2word = {i: w for w, i in word2idx.items()}

print("=== a. Data preparation ===")
print(f"Total tokens : {len(tokens)}")
print(f"Vocab size   : {V}")
print(f"Vocabulary   : {vocab}\n")
```

```
=== a. Data preparation ===
Total tokens : 79
Vocab size   : 44
Vocabulary   : ['a', 'amounts', 'and', 'are', 'at', 'away', 'barks', 'brow
n', 'by', 'capture', 'cat', 'computers', 'context', 'dog', 'embeddings', '
fox', 'friends', 'from', 'helps', 'human', 'in', 'its', 'jumps', 'language
', 'large', 'lazy', 'learn', 'learns', 'likes', 'meaning', 'model', 'natur
al', 'of', 'over', 'predicting', 'processing', 'quick', 'runs', 'text', 't
he', 'understand', 'vectors', 'word', 'words']
```

In [3]:
```
# ---------------------------------------------------------------
```

```
# b. GENERATE TRAINING DATA  (context words -> target word)
# ------------------------------------------------------------------
WINDOW = 2   # words on each side of the target


def generate_training_data(tokens, window):
    data = []
    for i, target in enumerate(tokens):
        context = [tokens[j]
                   for j in range(max(0, i - window), min(len(tokens), i
                   if j != i]
        data.append((context, target))
    return data


def one_hot(idx, size):
    v = np.zeros(size)
    v[idx] = 1
    return v


pairs = generate_training_data(tokens, WINDOW)
# X: (context indices list), y: target index
X = [[word2idx[w] for w in ctx] for ctx, _ in pairs]
y = [word2idx[t] for _, t in pairs]

print("=== b. Training data (first 5 samples) ===")
for ctx, tgt in pairs[:5]:
    print(f"  context={ctx}  ->  target='{tgt}'")
print(f"Total training samples: {len(pairs)}\n")
```

```
=== b. Training data (first 5 samples) ===
  context=['quick', 'brown']  ->  target='the'
  context=['the', 'brown', 'fox']  ->  target='quick'
  context=['the', 'quick', 'fox', 'jumps']  ->  target='brown'
  context=['quick', 'brown', 'jumps', 'over']  ->  target='fox'
  context=['brown', 'fox', 'over', 'the']  ->  target='jumps'
Total training samples: 79
```

In [5]:

```
# ------------------------------------------------------------------
# c. TRAIN MODEL
# ------------------------------------------------------------------
N = 10            # embedding dimension
EPOCHS = 500
LR = 0.05

W1 = np.random.randn(V, N) * 0.1    # input  -> hidden (word embeddings)
W2 = np.random.randn(N, V) * 0.1    # hidden -> output


def softmax(u):
    e = np.exp(u - np.max(u))
    return e / e.sum()


def forward(context_idx):
    # average the one-hot context vectors == average the rows of W1
    x_avg = np.mean([one_hot(i, V) for i in context_idx], axis=0)
    h = x_avg @ W1                  # hidden layer (N,)
```

```
    u = h @ W2                      # scores (V,)
    y_hat = softmax(u)              # probabilities (V,)
    return x_avg, h, y_hat


print("=== c. Training ===")
loss_history = []
for epoch in range(1, EPOCHS + 1):
    total_loss = 0.0
    for ctx_idx, tgt_idx in zip(X, y):
        x_avg, h, y_hat = forward(ctx_idx)

        # cross-entropy loss
        total_loss += -np.log(y_hat[tgt_idx] + 1e-9)

        # backprop
        e = y_hat.copy()
        e[tgt_idx] -= 1                         # dL/du
        dW2 = np.outer(h, e)                    # (N, V)
        dh = W2 @ e                             # (N,)
        dW1 = np.outer(x_avg, dh)               # (V, N)

        W2 -= LR * dW2
        W1 -= LR * dW1

    loss_history.append(total_loss / len(X))
    if epoch % 50 == 0 or epoch == 1:
        print(f"  Epoch {epoch:4d}/{EPOCHS}  loss = {loss_history[-1]:.4f
```

```
=== c. Training ===
  Epoch    1/500  loss = 3.7857
  Epoch   50/500  loss = 2.3024
  Epoch  100/500  loss = 0.8012
  Epoch  150/500  loss = 0.3052
  Epoch  200/500  loss = 0.1727
  Epoch  250/500  loss = 0.1174
  Epoch  300/500  loss = 0.0920
  Epoch  350/500  loss = 0.0791
  Epoch  400/500  loss = 0.0716
  Epoch  450/500  loss = 0.0668
  Epoch  500/500  loss = 0.0634
```

In [6]:

```
# ------------------------------------------------------------------
# d. OUTPUT
# ------------------------------------------------------------------
print("\n=== d. Output ===")


def predict(context_words, top_k=3):
    idxs = [word2idx[w] for w in context_words]
    _, _, y_hat = forward(idxs)
    top = np.argsort(y_hat)[::-1][:top_k]
    return [(idx2word[i], float(y_hat[i])) for i in top]


def most_similar(word, top_k=3):
    emb = W1 / (np.linalg.norm(W1, axis=1, keepdims=True) + 1e-9)
    sims = emb @ emb[word2idx[word]]
    best = np.argsort(sims)[::-1][1:top_k + 1]
    return [(idx2word[i], float(sims[i])) for i in best]
```

```
tests = [["the", "quick", "fox", "jumps"],
         ["the", "lazy", "the", "barks"],
         ["word", "capture", "meaning", "of"]]
for ctx in tests:
    print(f"Context {ctx} -> predicted: {predict(ctx)}")

print("\nEmbedding of 'fox':")
print(np.round(W1[word2idx['fox']], 3))

print("\nMost similar words:")
for w in ["fox", "language", "dog"]:
    print(f"  {w}: {[(a, round(b, 3)) for a, b in most_similar(w)]}")

print(f"\nFinal loss: {loss_history[-1]:.4f} (started at {loss_history[0]
```

```
=== d. Output ===
Context ['the', 'quick', 'fox', 'jumps'] -> predicted: [('brown', 0.986903
2810365875), ('over', 0.009913551160095123), ('dog', 0.00147377409452579
2)]
Context ['the', 'lazy', 'the', 'barks'] -> predicted: [('dog', 0.997624724
8325716), ('fox', 0.0021890490533675016), ('barks', 9.9201554000431e-05)]
Context ['word', 'capture', 'meaning', 'of'] -> predicted: [('by', 0.51848
80794469059), ('amounts', 0.28545026131717455), ('words', 0.11546024775102
882)]

Embedding of 'fox':
[-1.461  2.256 -1.312  3.183 -3.545  2.514  3.648  2.106 -2.06   0.77 ]

Most similar words:
  fox: [('embeddings', 0.749), ('cat', 0.647), ('dog', 0.579)]
  language: [('by', 0.619), ('away', 0.547), ('runs', 0.452)]
  dog: [('embeddings', 0.739), ('fox', 0.579), ('cat', 0.556)]

Final loss: 0.0634 (started at 3.7857)
```

In [ ]: